

VeganTrack

Tu nutrición plant-based, bajo control

Documento de Arquitectura y Roadmap

Stack: React + Vite + Tailwind CSS + Supabase

Marzo 2026 • v1.0

1. Visión del Producto

VeganTrack es una PWA de tracking nutricional diseñada específicamente para personas que siguen una alimentación plant-based. A diferencia de MyFitnessPal u otras apps genéricas, VeganTrack prioriza la precisión de datos mediante la integración directa con OpenFoodFacts y ofrece funcionalidades específicas para el público vegano: alternativas vegetales a productos escaneados, alertas de micronutrientes críticos en dietas plant-based (B12, hierro, omega-3, zinc, yodo) y un enfoque basado en datos, no en opiniones.

Propuesta de valor diferencial

- Enfoque 100% plant-based con sugerencias de alternativas veganas
- Alertas inteligentes de micronutrientes críticos para veganos (B12, hierro, zinc, omega-3, yodo, vitamina D)
- Datos reales de OpenFoodFacts, no bases de datos genéricas con errores
- Filosofía DATOS > OPINIÓN aplicada a la nutrición
- Diseño mobile-first, rápido y sin fricción

2. Stack Tecnológico

El stack elegido maximiza la velocidad de desarrollo, minimiza costes y aprovecha la experiencia previa con GymTracker, el inventario industrial y la app de Dragon Boat.

Capa	Tecnología	Justificación
Frontend	React 18 + Vite	Build ultra-rápido, HMR instantáneo, ecosistema maduro
Estilos	Tailwind CSS v3	Utility-first, consistencia visual, tamaño final mínimo con purge
Estado	Zustand	Ligero, sin boilerplate, compatible con persist middleware para offline
Backend/DB	Supabase	Auth, PostgreSQL, Row Level Security, Realtime, Storage — todo incluido gratis hasta escalar
API Nutrición	OpenFoodFacts	Base de datos abierta y gratuita, ideal para productos plant-based europeos
Barcode Scanner	html5-qrcode / QuaggaJS	Acceso nativo a cámara desde PWA, sin dependencias nativas
Charts	Recharts	Componentes React nativos, responsive, fácil de personalizar
Deploy	Vercel	CI/CD automático desde GitHub, SSL, preview deploys por PR
APK (futuro)	PWABuilder	Empaquetado como app Android sin código nativo, ya probado con GymTracker

3. Arquitectura de la Aplicación

3.1 Estructura de carpetas

src/

- components/ — Componentes reutilizables (UI atoms, molecules)
- components/ui/ — Botones, inputs, cards, modals (design system)
- components/scanner/ — BarcodeScanner, ScanResult
- components/food/ — FoodCard, NutrientBar, MealSection
- components/charts/ — MacroChart, MicroChart, ProgressRing
- features/ — Módulos por funcionalidad
- features/diary/ — Log diario de comidas
- features/search/ — Búsqueda y escáner de alimentos
- features/dashboard/ — Dashboard y gráficas de progreso
- features/profile/ — Perfil, objetivos, configuración
- hooks/ — Custom hooks (useScanner, useNutrition, useDebounce)
- lib/ — Clientes externos (supabase.js, openfoodfacts.js)
- stores/ — Zustand stores (diaryStore, userStore, searchStore)
- utils/ — Helpers (nutrient calculations, date formatting)
- types/ — TypeScript interfaces y types

3.2 Flujo de datos principal

El flujo central de la app sigue este patrón:

1. El usuario escanea un código de barras o busca un alimento por texto
2. La app consulta OpenFoodFacts API (GET <https://world.openfoodfacts.org/api/v2/product/{barcode}>)
3. Si el producto existe, se muestran los datos nutricionales completos (por 100g y por porción)
4. Si el producto NO es vegano, se sugieren alternativas plant-based de la misma categoría
5. El usuario selecciona cantidad y añade al log diario (desayuno/comida/cena/snacks)
6. Los datos se guardan en Supabase (online) y en Zustand persist (offline cache)
7. El dashboard se actualiza en tiempo real con macros y micros del día

3.3 Offline-first strategy

La app funciona offline-first usando Zustand persist con localStorage como capa de cache. Cuando hay conexión, los datos se sincronizan con Supabase. Los productos consultados previamente se cachean localmente para evitar llamadas repetidas a OpenFoodFacts. Un Service Worker (generado por vite-plugin-pwa) gestiona el caching de assets estáticos.

4. Esquema de Base de Datos (Supabase / PostgreSQL)

4.1 Tabla: profiles

Columna	Tipo	Descripción
id	uuid (PK)	Referencia a auth.users.id
display_name	text	Nombre visible
height_cm	numeric	Altura en cm
weight_kg	numeric	Peso actual en kg

birth_date	date	Fecha de nacimiento
activity_level	text	sedentary / light / moderate / active / very_active
goal	text	cut / maintain / bulk
calorie_target	integer	Objetivo calórico diario (calculado o manual)
protein_target_g	integer	Objetivo proteína (g)
carbs_target_g	integer	Objetivo carbohidratos (g)
fat_target_g	integer	Objetivo grasas (g)
created_at	timestampz	Fecha de creación

4.2 Tabla: food_log

Columna	Tipo	Descripción
id	uuid (PK)	ID único del registro
user_id	uuid (FK)	Referencia a profiles.id
date	date	Fecha del registro
meal_type	text	breakfast / lunch / dinner / snack
food_name	text	Nombre del alimento
barcode	text	Código de barras (nullable)
brand	text	Marca del producto (nullable)
serving_size_g	numeric	Tamaño de porción consumida en gramos
calories	numeric	Calorías de la porción
protein_g	numeric	Proteínas (g)
carbs_g	numeric	Carbohidratos (g)
fat_g	numeric	Grasas (g)
fiber_g	numeric	Fibra (g)
sugar_g	numeric	Azúcares (g)
saturated_fat_g	numeric	Grasas saturadas (g)
sodium_mg	numeric	Sodio (mg)
is_vegan	boolean	Indicador vegano según OpenFoodFacts
image_url	text	URL de imagen del producto
created_at	timestampz	Timestamp de creación

4.3 Tabla: food_cache

Cache local de productos consultados en OpenFoodFacts para reducir llamadas a la API y permitir funcionamiento offline.

Columna	Tipo	Descripción
barcode	text (PK)	Código de barras del producto

data	jsonb	Datos completos del producto de OpenFoodFacts
fetches_at	timestampz	Fecha de última consulta (para invalidar cache)

4.4 Tabla: micronutrients_log

Registro detallado de micronutrientes críticos para dietas plant-based. Se calcula y almacena al añadir alimentos al log.

Columna	Tipo	Descripción
id	uuid (PK)	ID único
user_id	uuid (FK)	Referencia a profiles.id
date	date	Fecha
vitamin_b12_mcg	numeric	Vitamina B12 (µg)
iron_mg	numeric	Hierro (mg)
zinc_mg	numeric	Zinc (mg)
calcium_mg	numeric	Calcio (mg)
omega3_g	numeric	Omega-3 (g)
iodine_mcg	numeric	Yodo (µg)
vitamin_d_mcg	numeric	Vitamina D (µg)
selenium_mcg	numeric	Selenio (µg)

4.5 Tabla: custom_foods

Alimentos personalizados creados por el usuario (recetas propias, alimentos que no están en OpenFoodFacts).

Columna	Tipo	Descripción
id	uuid (PK)	ID único
user_id	uuid (FK)	Referencia a profiles.id
name	text	Nombre del alimento/receta
calories_per_100g	numeric	Calorías por 100g
protein_per_100g	numeric	Proteínas por 100g
carbs_per_100g	numeric	Carbohidratos por 100g
fat_per_100g	numeric	Grasas por 100g
is_vegan	boolean	Indicador vegano
created_at	timestampz	Fecha de creación

5. Integración con OpenFoodFacts

5.1 Endpoints principales

- Búsqueda por barcode: GET <https://world.openfoodfacts.org/api/v2/product/{barcode}>
- Búsqueda por texto: GET https://world.openfoodfacts.org/cgi/search.pl?search_terms={query}&json=1
- Filtrar veganos: añadir `&tagtype_0=labels&tag_contains_0=contains&tag_0=en:vegan`

5.2 Campos clave del response

- `product.product_name` — Nombre del producto
- `product.brands` — Marca
- `product.nutriments` — Objeto con todos los valores nutricionales
- `product.nutriments.energy-kcal_100g` — Calorías por 100g
- `product.nutriments.proteins_100g` — Proteínas por 100g
- `product.nutriments.carbohydrates_100g` — Carbohidratos por 100g
- `product.nutriments.fat_100g` — Grasas por 100g
- `product.labels_tags` — Array que puede contener 'en:vegan'
- `product.image_front_url` — Imagen del producto
- `product.categories_tags` — Categorías (para buscar alternativas veganas)

5.3 Estrategia de alternativas veganas

Cuando un producto escaneado NO es vegano (no contiene 'en:vegan' en `labels_tags`), la app buscará productos veganos en la misma categoría principal usando el endpoint de búsqueda con filtro de label vegano. Ejemplo: si se escanea un yogur lácteo (categoría 'en:yogurts'), buscará 'en:yogurts' + label 'en:vegan' para mostrar alternativas vegetales.

6. Roadmap de Desarrollo (Fases)

Fase 1 — Fundamentos (Semana 1-2)

Objetivo: Proyecto configurado, auth funcionando, estructura base lista.

1. Inicializar proyecto: `npm create vite@latest vegantrack -- --template react-ts`
2. Instalar dependencias: `tailwindcss`, `@supabase/supabase-js`, `zustand`, `react-router-dom`
3. Configurar Supabase: crear proyecto, definir tablas (profiles), configurar Row Level Security
4. Implementar Auth: registro/login con email (Supabase Auth), protección de rutas
5. Crear layout base: bottom navigation (Diary, Search, Dashboard, Profile), header
6. Perfil de usuario: formulario de onboarding (peso, altura, objetivo, nivel actividad)
7. Calculadora TDEE automática basada en datos del perfil (Harris-Benedict o Mifflin-St Jeor)

Fase 2 — Core: Scanner + Search + Log (Semana 3-4)

Objetivo: Se pueden buscar alimentos, escanear productos y añadirlos al log diario.

8. Integrar `html5-qrcode` para el escáner de códigos de barras
9. Crear `lib/openfoodfacts.ts` con funciones de búsqueda por barcode y por texto
10. Pantalla de resultado de escaneo: mostrar nombre, imagen, macros, micros, label vegano
11. Búsqueda manual con `debounce` (300ms) y resultados paginados
12. Selector de cantidad (gramos / porción predefinida) con cálculo en tiempo real
13. Añadir al log diario: seleccionar `meal_type`, guardar en Supabase + cache local
14. Vista de log diario: lista de comidas del día agrupadas por `meal_type`
15. Crear tabla `food_cache` para almacenar productos consultados

Fase 3 — Dashboard + Gráficas (Semana 5-6)

Objetivo: Dashboard visual completo con progreso diario y semanal.

16. Progress ring principal: calorías consumidas vs objetivo
17. Barras de macronutrientes: proteína, carbo, grasas con % del objetivo
18. Gráfica semanal de calorías (Recharts AreaChart)
19. Panel de micronutrientes críticos: B12, hierro, zinc, calcio, omega-3, vitamina D
20. Alertas visuales cuando un micronutriente crítico está por debajo del 50% de la RDA
21. Historial: vista de calendario para navegar entre días

Fase 4 — Alternativas Veganas + Polish (Semana 7-8)

Objetivo: Diferenciación con funcionalidad vegana y pulido de UX.

22. Detección automática de producto no vegano al escanear
23. Búsqueda de alternativas veganas en la misma categoría de OpenFoodFacts
24. Card de alternativa: nombre, marca, comparación nutricional lado a lado
25. Alimentos personalizados: formulario para crear custom_foods (recetas propias)
26. Alimentos frecuentes/favoritos para acceso rápido
27. Configurar vite-plugin-pwa: Service Worker, manifest.json, iconos
28. Testing manual completo y fix de bugs

Fase 5 — Lanzamiento (Semana 9-10)

Objetivo: App lista para usuarios reales.

29. Deploy en Vercel con dominio personalizado
30. Empaquetar APK con PWABuilder para Android
31. Landing page simple con propuesta de valor
32. Integración con cuentas de @VeganoEficaz y @manual__verde para distribución
33. Feedback loop: formulario de feedback in-app
34. Monitorizar uso con Vercel Analytics o Plausible

7. Funcionalidades Futuras (Post-MVP)

- Recetas completas: crear recetas con múltiples ingredientes y cálculo nutricional automático
 - Meal planning semanal con generación de lista de compra
 - Integración con GymTracker para correlacionar nutrición con rendimiento en el gym
 - Export de datos a CSV/PDF para compartir con nutricionista
 - Social features: compartir meals, seguir a otros usuarios veganos
 - Gamificación: rachas, logros, objetivos semanales
 - API propia de alimentos veganos curada por la comunidad
 - Modelo freemium: features básicas gratis, dashboard avanzado y micros detallados en premium
-

8. Seguridad y Row Level Security

Todas las tablas de Supabase deben tener Row Level Security (RLS) activado. Las políticas deben garantizar que cada usuario solo puede leer y escribir sus propios datos.

Políticas RLS clave

- profiles: SELECT/UPDATE WHERE auth.uid() = id
- food_log: SELECT/INSERT/UPDATE/DELETE WHERE auth.uid() = user_id
- micronutrients_log: SELECT/INSERT WHERE auth.uid() = user_id
- custom_foods: SELECT/INSERT/UPDATE/DELETE WHERE auth.uid() = user_id
- food_cache: SELECT/INSERT para todos los usuarios autenticados (cache compartida)

9. Métricas de Éxito (KPIs)

Métrica	Objetivo MVP	Objetivo 6 meses
Usuarios registrados	50	500+
DAU (Daily Active Users)	10	100+
Logs/día por usuario activo	3+	5+
Retención D7	30%	50%+
Productos escaneados/día	20	200+
Lighthouse PWA score	90+	95+

VeganTrack — DATOS > OPINIÓN

Tu nutrición plant-based, respaldada por ciencia.